Association, Aggregation, Composition

While **Inheritance** focuses on the **"Is-A"** relationship (a child is a version of its parent), these concepts represent the **"Has-A"** relationship, showing how classes connect or contain one another without a parent-child hierarchy.

1. Association ("Uses-A" or "Knows-A"): It simply means that two independent objects communicate or use each other. There is no ownership, and both objects have their own independent lifecycles.

- **The Concept:** Object **A** and Object **B** interact, but neither "belongs" to the other. If one is deleted, the other is perfectly fine.
- **Analogy:** A **Doctor** and a **Patient**. If the doctor retires, the patient still exists.
- **In Code:** Usually represented by a class passing another class as a parameter in a method.

```
class Driver { /* ... */ };

class Car {
  void assignDriver(Driver *d) { /* Car uses a Driver */ }
};
```

2. Aggregation (Weak "Has-A"): A specialized, stronger form of Association. It represents a **"Has-A"** or **"Part-Of"** relationship, but with **weak ownership**. The child object can still exist completely independently of the parent object.

- **The Concept:** The parent contains the child, but the child's lifecycle is not tied to the parent's lifecycle.
- **Analogy:** A **University** and a **Professor**. The university **"has"** professors. However, if the university closes down, the professors don't extinct; they just go get jobs at a different university.
- **In Code:** Usually represented by passing an existing object into a class's constructor (Dependency Injection). The child object is created outside the parent.

```
class Tire { /* ... */ };

class Car {
    Tire* spare; // The tire is passed in; if the car is destroyed, Tire survives.
public:
    Car(Tire* t) : spare(t) {}
};
```

3. Composition (Strong "Has-A"): A strictest form of relationship. It represents a "Has-A" relationship with **strong ownership** (sometimes called a **"death relationship").** The child object cannot exist independently of the parent.

- **The Concept:** The parent strictly owns the child. If the parent object is destroyed, the child object is destroyed with it.
- **Analogy:** A **House** and a **Room**. A house **"has"** rooms. If you completely demolish the house, the rooms no longer exist. You cannot move a **"room"** to a different house.
- **In Code:** Usually represented by the parent class creating the child object inside it. The child object is entirely managed by the parent.

```
class Engine { /* ... */ };

class Car {
    Engine engine; // Engine is created inside Car; if Car is destroyed, Engine is too.
};
```